

Kafka Interview Guide

1. Kafka basics

Producer: sends records to topics, chooses target partition, batches data, and uses acks for reliability.

Consumer: reads from partitions with offsets, works in consumer groups, and each partition is processed by one consumer in a group.

Topic: logical stream of events. Partition: ordered log segment with offsets and parallelism. Broker: Kafka server process.

Storage: data is persisted to disk and replicated across brokers for durability and recovery.

2. Zookeeper working

Zookeeper maintains broker metadata, election state, membership, and leadership coordination in older Kafka deployments.

Leader election: controller watches broker liveness and picks a new leader from the ISR when a leader fails.

Follower election: followers replicate leader log and become eligible when they are caught up and in sync.

3. KRaft working

KRaft removes ZooKeeper and uses Raft-based consensus for metadata quorum and leadership election.

Leader election: a quorum elects a leader using term numbers and heartbeats. Followers replicate metadata logs.

Modern Kafka uses KRaft for simpler operations and no external coordination service.

4. Kafka brokers

Brokers receive producer writes, serve consumer reads, store partitions, replicate data, and maintain cluster state.

Important broker configs: listeners, advertised.listeners, log.dirs, num.io.threads, log.retention.hours, and replica.fetch.max.bytes.

5. Interview Q&A

Q: What is Kafka? A: Distributed streaming platform for real-time event delivery, durability, and scalable data pipelines.

Q: What is consumer group? A: Set of consumers that together read a topic, each partition handled by one consumer in the group.

Q: What is ISR? A: In-Sync Replicas, replicas caught up sufficiently to become a leader.

Q: Why use Kafka? A: High throughput, decoupling, replayability, fault tolerance, and horizontal scaling.

Q: What is offset? A: Unique position of a record in a partition.

6. Scenario-based questions

Scenario: consumer lag is rising. Answer: scale consumers, add partitions, fix processing bottlenecks, and monitor lag.

Scenario: broker down. Answer: followers continue serving reads if replication is healthy and leadership fails over.

Scenario: data loss. Answer: verify acks=all, replication factor, min.insync.replicas, and retries.

Scenario: duplicate reads. Answer: Kafka is at-least-once by default, so consumers need deduplication logic.

7. Do's and Dont's

Do: use replication factor 3, monitor lag, keep partitions aligned to consumer parallelism, and use acks=all for critical topics.

Dont: ignore under-replicated partitions, run single-broker production, or misconfigure topic partitions.

8. Producer, consumer, topic configs

Producer: bootstrap.servers, acks, retries, linger.ms, batch.size, compression.type, enable.idempotence.

Consumer: group.id, auto.offset.reset, enable.auto.commit, max.poll.records, session.timeout.ms, isolation.level.

Topic: num.partitions, replication.factor, retention.ms, cleanup.policy, min.insync.replicas.

9. Monitoring and Prometheus

Monitor: consumer lag, broker CPU, disk usage, replication lag, under-replicated partitions, bytes in and out, and request latency.

Prometheus scrape config: static_configs targets: [broker1:7071, broker2:7071, broker3:7071].

JMX exporter exposes Kafka metrics, which Grafana visualizes for performance and scalability.

Production tip: keep quorum, broker health, lag, and storage usage under constant watch.

This PDF is a compact summary of the Kafka guide created for interview preparation and production understanding.